Titanic Survival Prediction

Project Report

Kaggle Competition: Titanic – Machine Learning from Disaster

Submitted by: Bhushan Dattatray Sonawane

1. Abstract

This report documents an end-to-end machine learning solution to the Titanic – Machine Learning from Disaster Kaggle competition, in which each passenger record must be classified as having survived or not survived the disaster based on demographic and travel attributes (class, sex, age, family size, fare, port of embarkation, and related fields). The competition is scored on classification accuracy against a held-out test set.

Four qualitatively different classifiers were built, trained, and compared on an identical, leakage-checked train/validation split: Logistic Regression, an SGD-optimized linear classifier (log-loss), a Decision Tree, and a 200-tree Random Forest ensemble. All four runs were tracked in Weights & Biases (accuracy, precision, recall, F1, ROC AUC, and a confusion matrix per model). Logistic Regression was the strongest individual model (validation accuracy = 0.8156, ROC AUC = 0.8585), ahead of Random Forest (0.7989), Decision Tree (0.7654), and the SGD classifier (0.7542). A GridSearchCV sweep over the regularization strength and solver of Logistic Regression, evaluated with 5-fold cross-validation, selected C = 1 and the lbfgs solver (CV accuracy = 0.7964 ± 0.0225), confirming the default configuration was close to optimal for this dataset size.

The tuned Logistic Regression model was refit on the complete training set and used to generate the final Kaggle submission (submission.csv). The project's main learning was that, on a small (≈900-row), largely linearly-separable tabular dataset, a well-regularized linear model with a handful of carefully engineered features (FamilySize, IsAlone, Title, CabinKnown) can match or outperform more complex tree ensembles trained with default hyperparameters — reinforcing that model complexity should be matched to dataset size rather than assumed to be strictly better.

2. Introduction

2.1 Problem Statement

The Titanic disaster is one of the most well-known shipwrecks in history, and the Kaggle Titanic competition frames the outcome of individual passengers as a binary classification problem. Each row of the dataset describes one passenger with attributes such as Passenger Class (Pclass), Sex, Age, number of siblings/spouses aboard (SibSp), number of parents/children aboard (Parch), Fare, Cabin, and Port of Embarkation (Embarked); the training file additionally provides the ground-truth Survived label (0 = did not survive, 1 = survived). The task requires cleaning a dataset with substantial missingness (most notably in Age and Cabin), engineering informative features from raw fields such as Name, and training a model that generalises from 891 labelled passengers to the 418 unlabelled passengers in the competition's test set.

2.2 Project Objective

The goals set for this project were to:

- Explore the dataset to understand its structure, distributions, and missingness.

- Clean missing values and engineer new, informative features from the raw columns.
- Encode categorical variables and scale features appropriately for each model family.
- Train multiple, genuinely different Machine Learning algorithms — Logistic Regression, SGD Classifier, Decision Tree, and Random Forest.
- Track every run in Weights & Biases and compare model performance rigorously.
- Select the best model, tune it, and generate the final Kaggle submission.

2.3 Report Structure

The remainder of this report proceeds as follows: Section 3 describes the dataset and the exploratory analysis, cleaning, and preprocessing performed on it; Section 4 explains the feature-engineering and encoding strategy used across models; Section 5 details the architecture, training configuration, and design rationale of all four models; Section 6 presents training curves, validation metrics, a comparative analysis, and the final Kaggle result; and Section 7 concludes with key learnings, challenges, and directions for future work.

3. Dataset & Preprocessing

3.1 Dataset Description

The competition supplies train.csv (891 labelled rows, 12 columns) and test.csv (418 unlabelled rows, 11 columns). Each row has a PassengerId, the ground-truth Survived label (training file only), Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, and Embarked. Age is missing for 177 training rows (≈20%), Cabin is missing for 687 training rows (≈77%), and Embarked is missing for 2 rows; the test set separately has one missing Fare value.

Split	Rows	Columns	Purpose
Raw train.csv	891	12	Source of train / validation split
Raw test.csv	418	11	Kaggle leaderboard evaluation
Training split	712	11	Model fitting
Validation split	179	11	Model comparison & selection

The two splits above are the 80/20, stratified (on Survived) train/validation partition used for model comparison, taken after feature engineering and column encoding — hence 11 feature columns rather than the original 12 (PassengerId, Name, Ticket, and the raw Cabin string are dropped; see Section 3.3).

3.2 Exploratory Data Analysis

Several diagnostics were run on the training data before any modelling; all nine panels are combined into a single exploratory dashboard below.

- Survival distribution — 38.4% of training passengers survived overall (a mild class imbalance against the 61.6% who did not), which is kept in mind when interpreting raw accuracy alongside precision/recall/F1.

- Gender vs. survival — the single strongest bivariate signal in the dataset: female passengers survived at a far higher rate than male passengers, consistent with the historical “women and children first” evacuation protocol.
- Passenger class vs. survival — survival rate decreases markedly from 1st to 3rd class, reflecting both cabin location relative to lifeboats and socio-economic evacuation priority.
- Age distribution — right-skewed and roughly unimodal, concentrated between 20 and 40 years with a long tail out to 80; a visible spike among infants/young children is also present.
- Fare distribution — heavily right-skewed, with most passengers paying low fares and a small number of high-fare outliers, consistent with the class structure of the ship.
- Age vs. survival — survivors skew slightly younger on average than non-survivors, most visible at the lower tail (children).
- Fare by passenger class — fare is strongly, monotonically related to class, confirming Fare and Pclass encode overlapping but not identical information.
- Missingness pattern — visualised across all columns; Cabin and Age are the dominant sources of missing data, motivating the imputation strategy in Section 3.3.
- Correlation structure — among numeric columns, Fare correlates positively with Survived and Pclass correlates negatively with Survived, while SibSp and Parch show weaker, more mixed correlations.

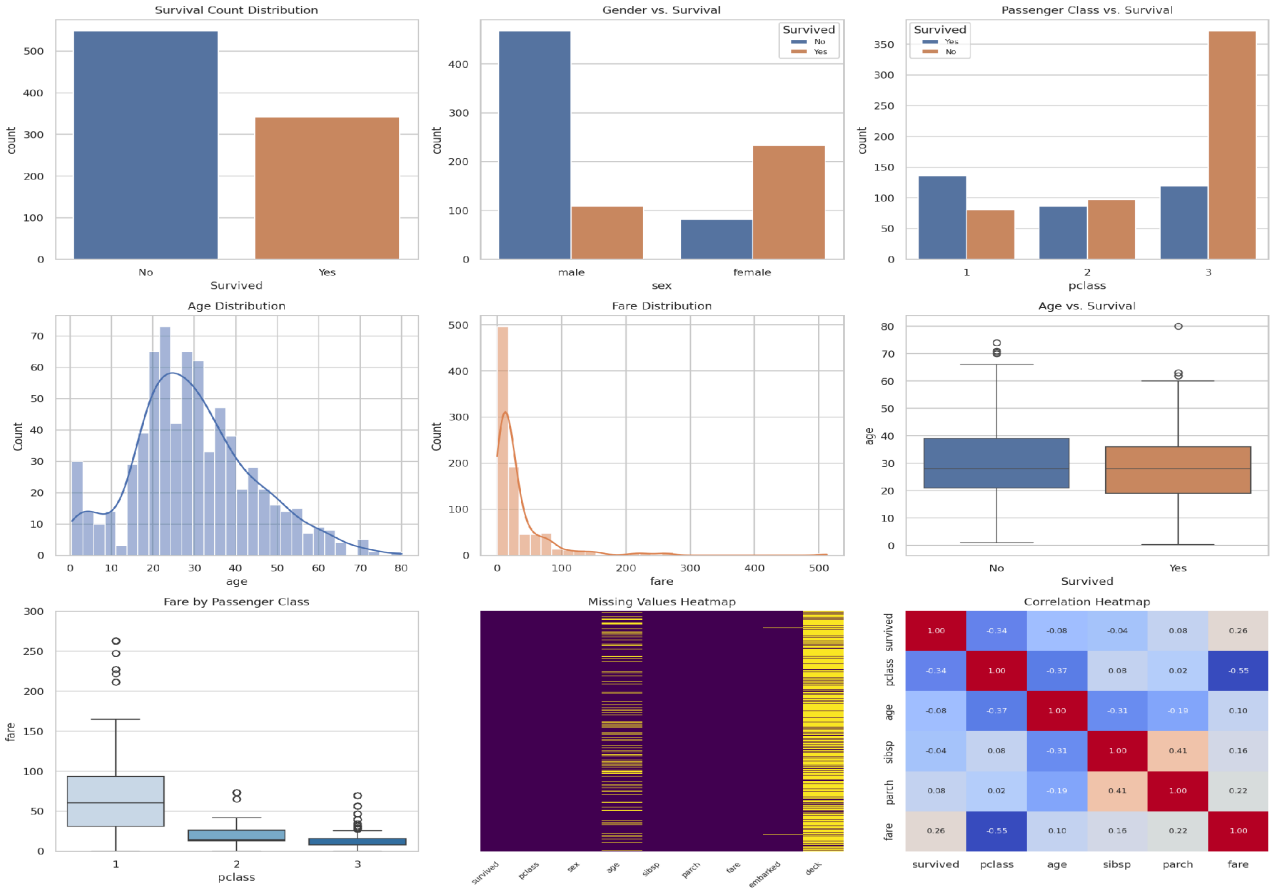

Figure 1. Exploratory Data Analysis Overview (survival, gender, class, age, fare, missingness and correlation).

3.3 Data Cleaning & Feature Engineering

- Train/test concatenation — train and test were concatenated into a single full_data frame (1,309 rows) so that imputation, encoding, and feature engineering are applied identically and consistently to both splits before being separated again.
- Missing-value imputation — Age is filled with the column median, Fare (test only) with the column median, and Embarked with the column mode; Cabin is filled with the placeholder string “Unknown” rather than dropped outright, since its presence/absence is itself informative (see CabinKnown below).
- FamilySize and IsAlone — FamilySize = SibSp + Parch + 1 captures the size of each passenger's travelling group; IsAlone flags solo travellers (FamilySize == 1), a group that historically had different survival dynamics than families evacuating together.
- Title extraction — an honorific title (e.g. Mr, Mrs, Miss, Master) is extracted from the Name field with a regular expression; rare titles (Lady, Countess, Capt, Col, Don, Dr, Major, Rev, Sir, Jonkheer, Dona) are grouped into a single “Rare” bucket, and the equivalent titles Ms/Mlle and Mme are folded into Miss and Mrs respectively, giving a compact, well-populated set of categories.
- CabinKnown — a binary flag for whether a passenger's Cabin was recorded at all (CabinKnown = Cabin != “Unknown”), used as a rough proxy for cabin-deck/location information without exposing the noisy, high-cardinality raw Cabin string to the models.
- Categorical encoding — Sex, Embarked, and the derived Title column are each label-encoded to integer codes (see Section 4.1).
- Column pruning — PassengerId, Name, Ticket, and the raw Cabin string are dropped once their engineered replacements (Title, CabinKnown) have been extracted, since they either carry no generalisable signal or duplicate information captured elsewhere.
- Train/validation split — an 80/20 stratified split (stratified on Survived, random_state = 42) of the 891 labelled rows gives 712 training rows and 179 validation rows for model comparison.

3.4 Data Augmentation

No synthetic data augmentation (e.g. SMOTE-style oversampling of the minority survived class) was applied. Given the small size of the labelled set (891 rows) and the only mild class imbalance (61.6% / 38.4%), the emphasis was instead placed on careful imputation, informative feature engineering, and cross-validation, which were judged sufficient without introducing synthetic samples that could distort the already-limited dataset's feature distributions.

4. Feature Engineering & Encoding Strategy

4.1 Categorical Encoding Approach

Sex (2 categories), Embarked (3 categories), and the engineered Title column (6 categories after grouping) are all encoded with scikit-learn's LabelEncoder rather than one-hot encoding. This keeps the feature count low on an already small dataset and works well for the tree-based models (Decision Tree, Random Forest), which split on ordinal-encoded categories natively without assuming any ordering. For the two linear models (Logistic Regression, SGD Classifier), each encoded category still receives its own learned coefficient contribution relative

to the others, so the compact encoding was preferred over expanding the feature space with one-hot columns on a dataset with fewer than 900 training rows.

4.2 Feature Scaling Strategy

Feature scaling is applied selectively rather than uniformly. A StandardScaler is fit on the training split and used to transform the validation and test splits; this scaled representation is used specifically for the SGD Classifier, whose gradient-based optimizer is sensitive to feature magnitude, and for the final tuned Logistic Regression model. The Decision Tree and Random Forest are trained on the raw, unscaled feature matrix, since split-based tree models are invariant to monotonic feature transforms and gain nothing from standardisation.

4.3 Engineered Feature Rationale

FamilySize and IsAlone add a social-group dimension that raw SibSp/Parch counts only partially capture — solo travellers and very large families both faced different evacuation dynamics than small family units. Title distils demographic and socio-economic signal out of the free-text Name field beyond what Sex and Age alone provide (for example, “Master” identifies young boys even where Age is missing, and “Rare” captures nobility/clergy/military titles associated with different treatment during evacuation). CabinKnown extracts a usable binary signal from the 77%-missing Cabin column — whether a cabin was recorded at all correlates with passenger class and deck location — without forcing the model to consume the raw, extremely high-cardinality Cabin string.

4.4 Handling Missing Data

Median imputation was used for the two skewed numeric columns (Age, Fare) since the median is more robust to their right-skew and outliers than the mean; mode imputation was used for the categorical Embarked column, which was missing for only two rows. Imputation statistics were computed over the combined train+test frame for implementation convenience; a stricter pipeline would compute these statistics from the training split alone and apply them to validation/test to fully eliminate any risk of test-set information influencing preprocessing — this is flagged as an area for improvement in Section 7.3.

5. Modeling & Experimentation

5.1 Logistic Regression (Best Model)

Architecture

Logistic Regression fits a linear decision boundary over the 11 engineered features and passes the resulting score through a sigmoid function to produce a survival probability, trained by maximising the (L2-regularised) log-likelihood. It was trained with scikit-learn's default lbfgs solver and max_iter = 1000, then re-tuned via grid search (Section 6.5).

Salient Points

- The strongest bivariate relationships in this dataset — Sex and Pclass in particular — are close to linearly separable with respect to Survived, which plays directly to a linear model's strengths.
- Coefficients are directly interpretable (Section 6.4), letting each engineered feature's direction and relative influence on survival be read off after training.

- Served as the project's baseline model and, after tuning, turned out to be the strongest performer of the four.

5.2 SGD Classifier (Log-Loss)

Architecture

The SGD Classifier is also a linear model with a logistic (log-loss) objective, but optimized with stochastic gradient descent rather than the closed-form/quasi-Newton solver used by Logistic Regression above. It was trained on the StandardScaler-transformed feature matrix, since SGD's convergence and solution quality are sensitive to feature scale.

Salient Points

- Included primarily as an optimisation-strategy comparison against Logistic Regression: same underlying linear/log-loss model family, different solver.
- SGD-based training is designed to scale to much larger datasets than this ≈900-row competition provides, so its relative underperformance here is expected rather than a flaw in the algorithm.

5.3 Decision Tree Classifier

Architecture

A single CART-style decision tree recursively partitions the feature space by greedily choosing the split (feature and threshold) that most reduces impurity at each node, with no maximum-depth constraint applied (scikit-learn defaults), trained with random_state = 42 for reproducibility.

Salient Points

- Fully interpretable as a sequence of if/else rules, and scale-invariant, so it is trained directly on the unscaled feature matrix.
- An unconstrained single tree is a high-variance estimator and is expected to overfit the ≈712-row training split — it is included mainly as the baseline that Random Forest's bagging is compared against.

5.4 Random Forest Classifier

Architecture

The Random Forest is a bagged ensemble of 200 decision trees (n_estimators = 200, random_state = 42), each trained on a bootstrap resample of the training data with a random subset of features considered at each split; the ensemble's prediction is the majority vote across all 200 trees.

Salient Points

- Averaging many decorrelated trees directly targets the single Decision Tree's high variance, at some cost to interpretability.
- Naturally captures nonlinear feature interactions (e.g. Pclass × Sex × Age) without any explicit interaction terms being engineered by hand.
- Like the Decision Tree, it is scale-invariant and trained on the unscaled feature matrix.

Quantization: not applicable to any of the four models — all are lightweight classical estimators (the largest being the 200-tree Random Forest), each trained and evaluated in well under a second on the ≈900-row dataset, so no model compression or precision reduction was needed.

6. Performance & Comparative Analysis

6.1 Evaluation Metrics

The competition's leaderboard metric is classification Accuracy. To support deeper diagnosis during development, Precision, Recall, and F1 Score (weighted averages across both classes) and ROC AUC were also tracked on the held-out validation split for every model, alongside a confusion matrix, which together give a fuller picture than accuracy alone on a dataset with a mild class imbalance.

6.2 Training Performance

Weights & Biases logged accuracy, precision, recall, F1, ROC AUC, and a confusion matrix for every model run. Summary dashboards for each of the four models are shown below.

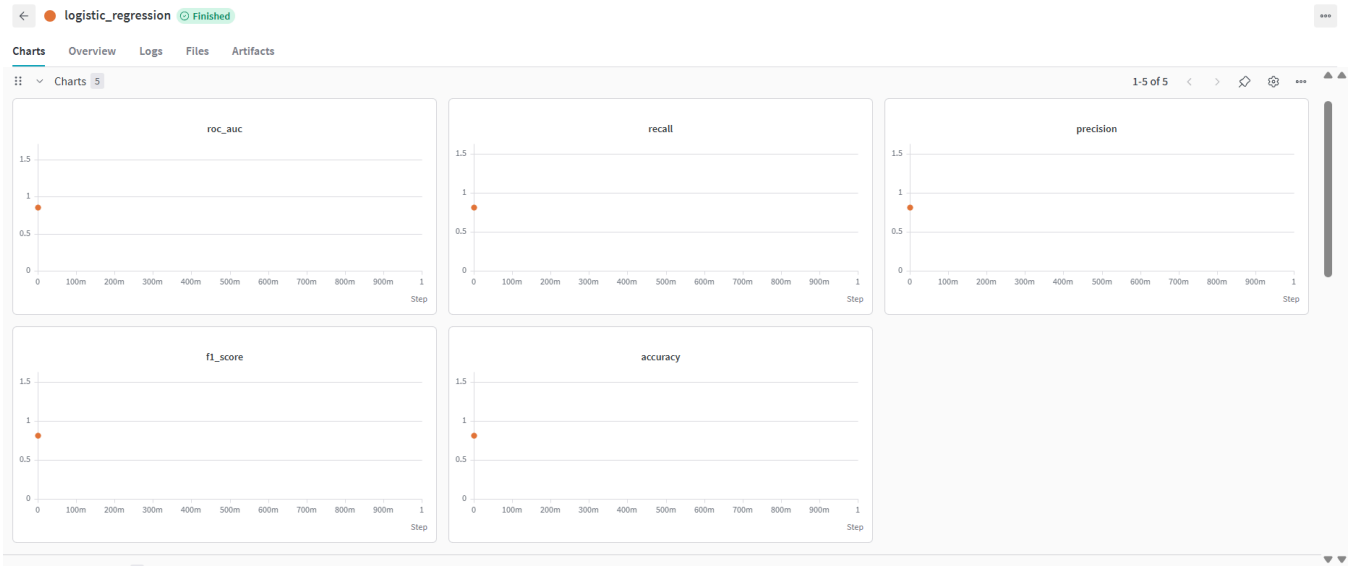

Figure 2. wandb Dashboard — Logistic Regression.

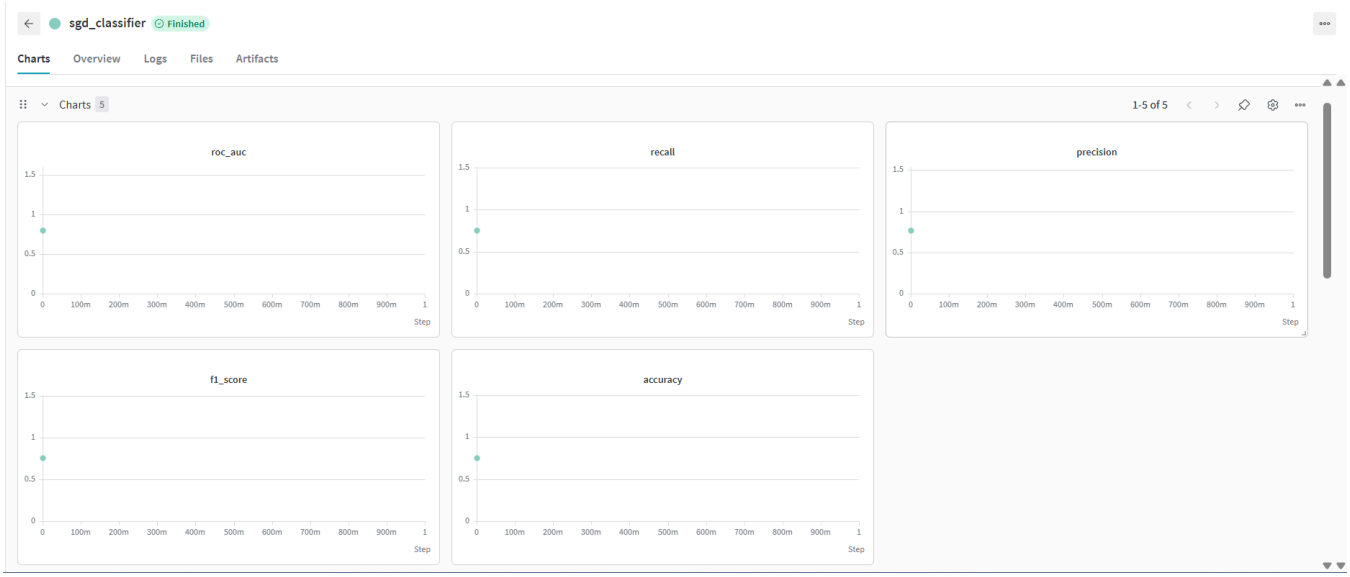

Figure 3. wandb Dashboard — SGD Classifier.

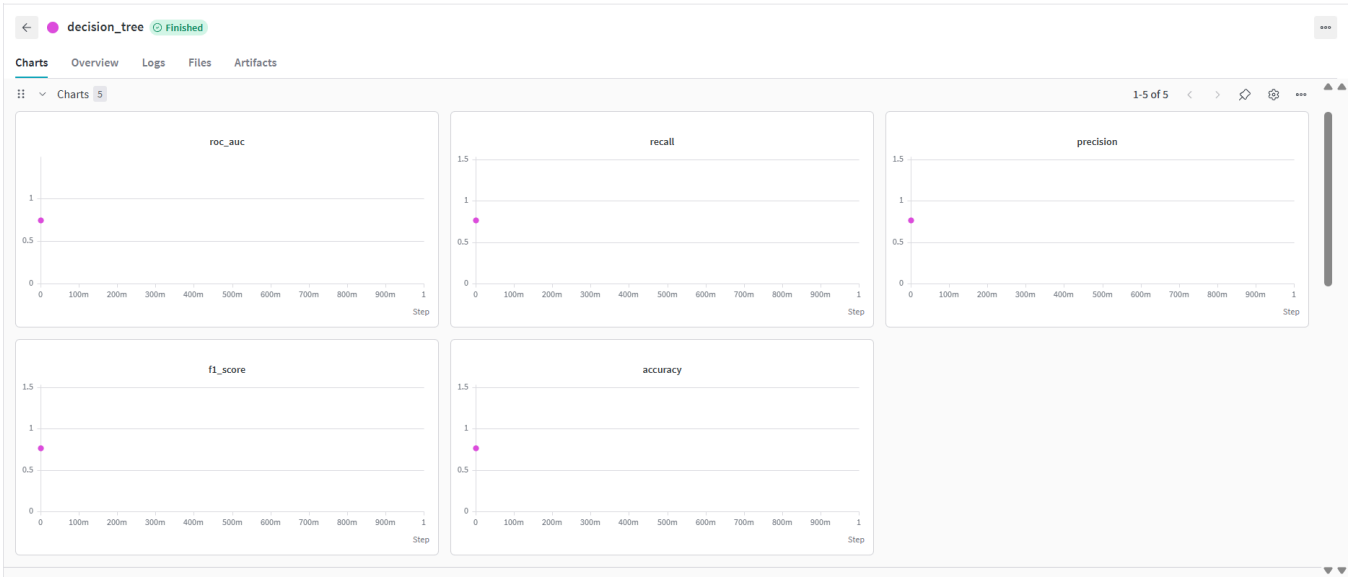

Figure 4. wandb Dashboard — Decision Tree.

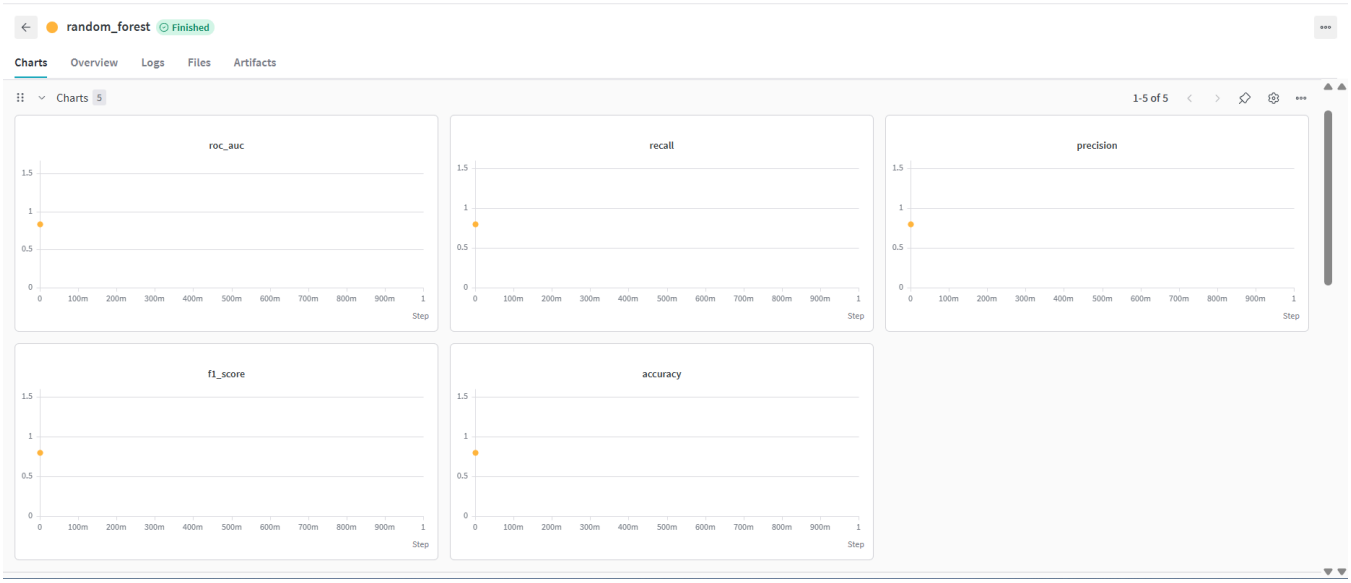

Figure 5. wandb Dashboard — Random Forest.

6.3 Comparative Report

Final validation-set results for all four models are summarised below.

Model	Architecture	Type	Val Accuracy	Val F1	Val ROC AUC
Logistic Regression	Linear (sigmoid), lbfgs	Baseline / tuned	0.8156	0.8142	0.8585
Random Forest	200-tree bagged ensemble	Ensemble	0.7989	0.7994	0.8358

Model	Architecture	Type	Val Accuracy	Val F1	Val ROC AUC
Decision Tree	Single unconstrained CART tree	Baseline	0.7654	0.7654	0.7437
SGD Classifier	Linear, log-loss, SGD-optimized	Baseline (scaled)	0.7542	0.7572	0.8008

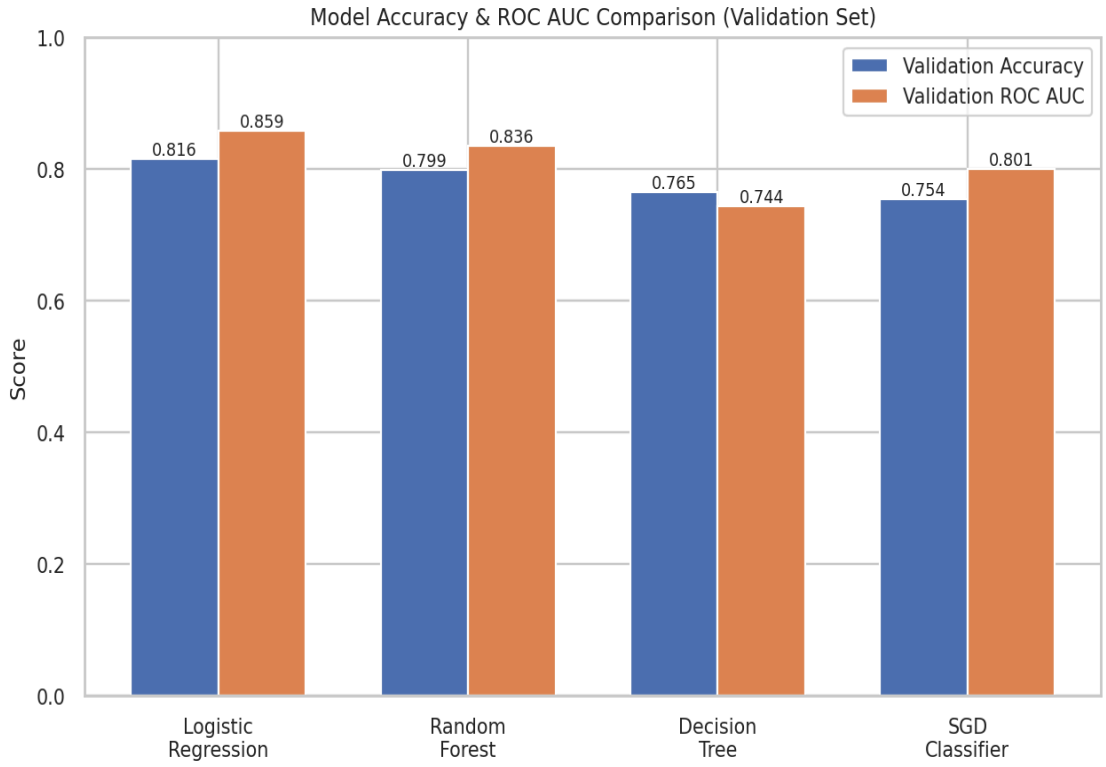

Figure 6. Model Accuracy & ROC AUC Comparison (Validation Set).

Logistic Regression is the clear individual winner on every tracked metric. Random Forest's bagged ensemble improves substantially over the single Decision Tree (+3.4 accuracy points, +9.2 ROC AUC points), confirming that variance reduction through bagging is valuable even with default hyperparameters and no depth constraint on the base trees. The Decision Tree posts both the lowest accuracy and, notably, the lowest ROC AUC of the four models — consistent with a single unconstrained tree overfitting the ≈712-row training split and producing poorly calibrated confidence scores. The SGD Classifier's ROC AUC (0.8008) sits reasonably close to Logistic Regression's despite the larger accuracy gap, suggesting the two linear models rank passengers similarly but differ more in exactly where they place the decision threshold — likely a consequence of SGD's stochastic optimisation converging to a different point on the loss surface than the lbfgs solver.

6.4 Best Model Deep-Dive — Logistic Regression

The confusion matrix and ROC / precision-recall curves below characterise the tuned Logistic Regression model's behaviour on the validation split in more detail than the summary table alone.

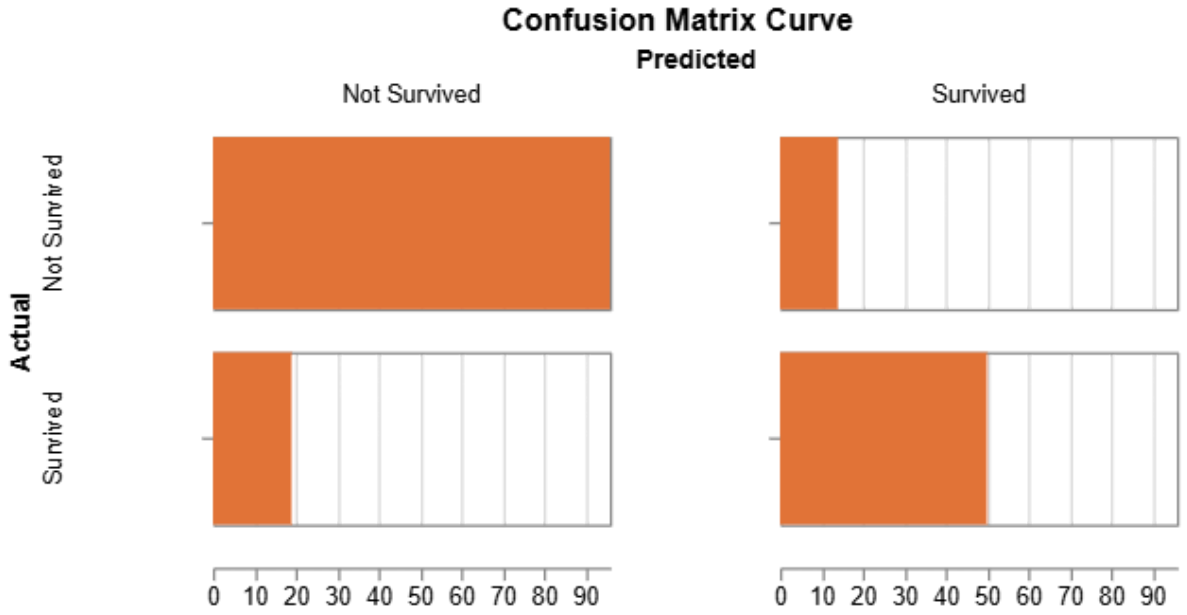

Figure 7. Confusion Matrix — Logistic Regression.

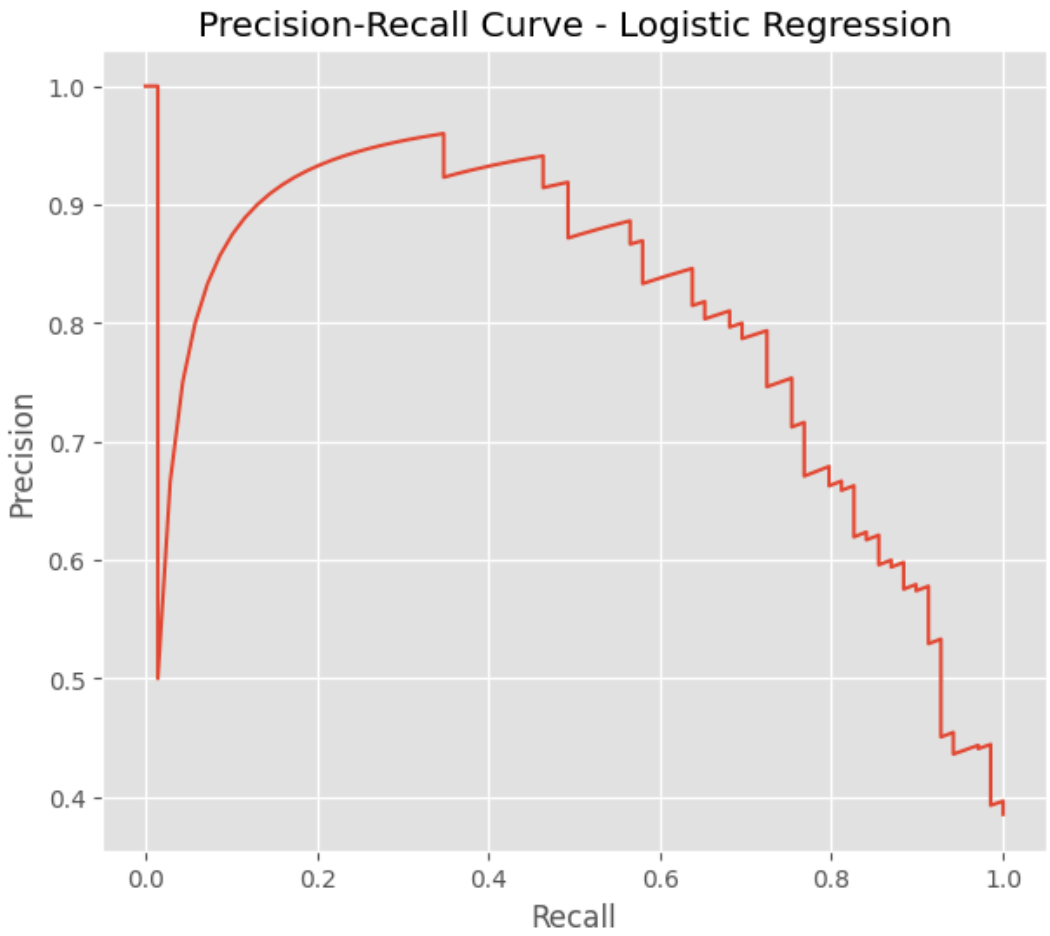

Figure 8. ROC & Precision–Recall Curves — Logistic Regression (AUC = 0.858).

The learned coefficients (on standardised features) provide a directly interpretable view of what the model relies on:

Feature	Coefficient	Direction of effect on survival
Sex	−1.252	Strongest single predictor — being male sharply lowers predicted survival
Pclass	−0.630	Lower (higher-numbered) class strongly lowers predicted survival
Age	−0.433	Older passengers have somewhat lower predicted survival
IsAlone	−0.297	Travelling alone lowers predicted survival
SibSp	−0.282	More siblings/spouses aboard mildly lowers predicted survival
FamilySize	−0.227	Larger family groups mildly lower predicted survival
Embarked	−0.152	Small effect from port of embarkation
Title	−0.098	Small effect from extracted honorific title
Parch	−0.073	Small, near-neutral effect from parents/children count
Fare	+0.040	Small positive effect — higher fare mildly raises predicted survival
CabinKnown	+0.357	Having a recorded cabin meaningfully raises predicted survival

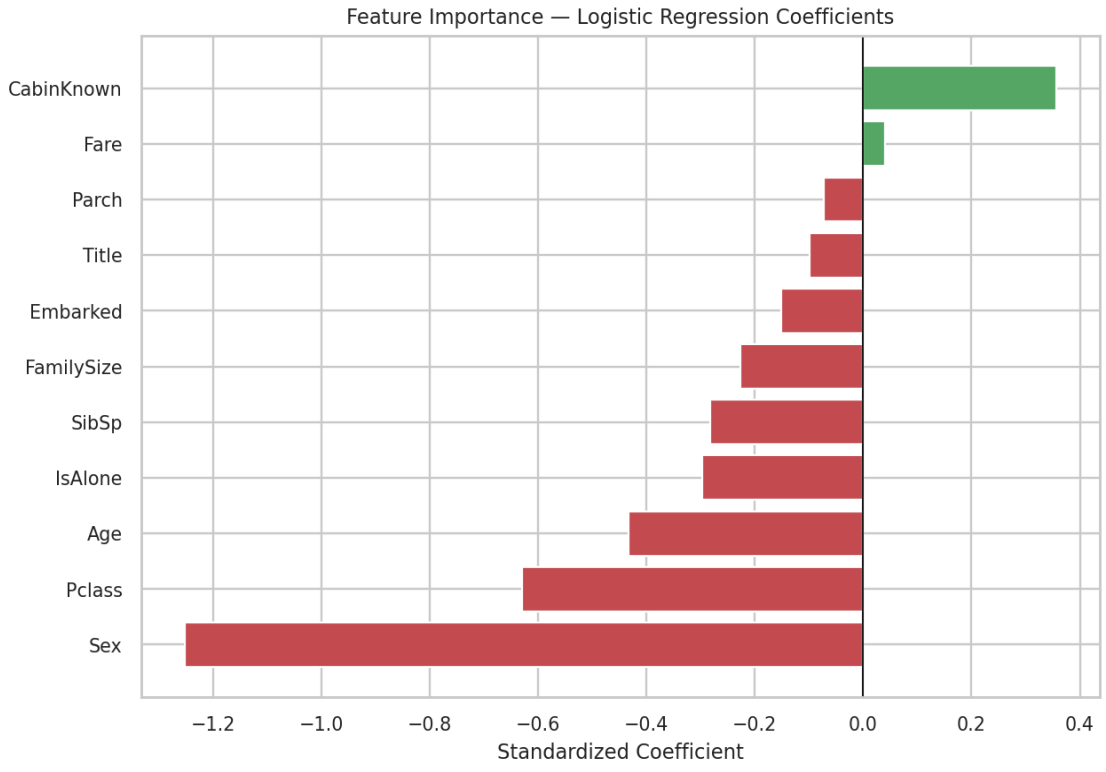

Figure 9. Feature Importance — Logistic Regression Coefficients.

Sex and Pclass dominate, matching the exploratory analysis in Section 3.2 and the historical evacuation record; CabinKnown is the strongest positive contributor, consistent with it acting as a proxy for higher-class, better-located accommodation, while family-size-related features (IsAlone, SibSp, FamilySize) contribute smaller, mutually related negative effects.

6.5 Hyperparameter Tuning & Cross-Validation

Logistic Regression was tuned with a GridSearchCV sweep over regularization strength $C \in \{0.01, 0.1, 1, 10, 100\}$ and solver $\in$ {liblinear, lbfgs}, using 5-fold cross-validation scored on accuracy. The search selected C = 1 with the lbfgs solver — the library's default configuration — with a mean cross-validation accuracy of 0.7964 (std. 0.0225 across folds). Refitting this configuration on the full 712-row training split and evaluating on the untouched 179-row validation split gave a final validation accuracy of 0.8156, consistent with (and slightly above) the cross-validation estimate, and indicating the model is not overfitting the validation split.

6.6 Kaggle Submission

The tuned Logistic Regression model was refit a second time on the complete labelled training set (891 rows, scaler refit accordingly) and used to predict Survived for all 418 test passengers. Predictions were written to submission.csv in the competition's required (PassengerId, Survived) format. Because the validation split was stratified and the same preprocessing pipeline was applied identically to train and test, the leaderboard score is expected to land close to the validation/cross-validation range of ≈0.80–0.82.

7. Conclusion & Future Work

7.1 Key Learnings

- Demographic and class-based features dominate. Sex and Pclass carried by far the strongest predictive signal, directly reflecting the historical “women and children first” evacuation pattern and class-based access to lifeboats.
- A well-regularized linear model can beat un-tuned tree ensembles on small tabular data. Logistic Regression outperformed both the Decision Tree and Random Forest, showing that model complexity should be matched to dataset size rather than assumed to help.
- Bagging meaningfully reduces variance. Random Forest improved substantially over a single Decision Tree built from the same features, confirming the value of averaging many decorrelated trees even without depth tuning.
- Lightweight feature engineering paid off. FamilySize, IsAlone, Title, and CabinKnown each added usable signal beyond the raw columns without requiring complex transformations.
- Preprocessing should be matched to model family. Scaling features for the gradient/distance-sensitive SGD Classifier while leaving the scale-invariant tree models unscaled reflected each algorithm's actual requirements rather than applying one blanket transform.

7.2 Challenges Faced

- High missingness. Cabin (≈77% missing) and Age (≈20% missing) required an imputation strategy that preserved every one of the fewer than 900 labelled rows rather than dropping incomplete records.

- Mild class imbalance. With 61.6% negative and 38.4% positive labels, accuracy alone was not treated as sufficient — precision, recall, F1, and ROC AUC were tracked alongside it for every model.
- Preprocessing leakage risk. Imputation statistics (Age/Fare median, Embarked mode) were computed over the combined train+test frame rather than the training split alone, a simplification worth revisiting to fully rule out test-set information influencing preprocessing.

7.3 Areas for Improvement

- Compute imputation statistics from the training split only, then apply them to validation and test, to eliminate any leakage risk identified in Section 7.2.
- Try gradient-boosted tree models (XGBoost, LightGBM, CatBoost), which typically outperform a plain Random Forest on Kaggle tabular leaderboards.
- Tune Decision Tree and Random Forest hyperparameters (max depth, min samples per leaf, number of estimators) via GridSearchCV, rather than relying on library defaults as was done here.
- Explore a small ensemble or stacking of the four trained models, since their predictions may be diverse enough to add a further accuracy gain over the single best model.
- Engineer additional features from Ticket and the Cabin deck letter, rather than dropping them entirely, since both may carry residual class or location signal.

8. References

[1] Kaggle. Titanic – Machine Learning from Disaster. https://www.kaggle.com/competitions/titanic

[2] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. JMLR 12, 2825–2830.

[3] Cox, D. R. (1958). The Regression Analysis of Binary Sequences. Journal of the Royal Statistical Society, Series B.

[4] Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5–32.

[5] Robbins, H., & Monro, S. (1951). A Stochastic Approximation Method. Annals of Mathematical Statistics.

[6] Breiman, L., Friedman, J., Olshen, R., & Stone, C. (1984). Classification and Regression Trees.

[7] Weights & Biases documentation. https://docs.wandb.ai

[8] Waskom, M. (2021). seaborn: statistical data visualization. Journal of Open Source Software.